Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

TUTORIAL DE JADE

PROGRAMACIÓN EN JADE PARA PRINCIPIANTES

USO RESTRINGIDO SEGÚN EL ACUERDO DE LICENCIA.

Última actualización: 30 de junio de 2009. JADE 3.7

Autores: Giovanni Caire (TILAB, anteriormente CSELT)

Copyright (C) 2000 CSELT S.p.A. Copyright
(C) 2001 TILab S.p.A. Copyright (C) 2002
TILab S.p.A. Copyright (C) 2003 TILab S.p.A.
Copyright (C) 2004 Telecom Italia S.p.A.
Derechos de autor (C) 2005 Telecom Italia
S.p.A. Derechos de autor (C) 2006 Telecom
Italia S.p.A. Derechos de autor (C) 2007
Telecom Italia S.p.A. Derechos de autor (C)
2008 Telecom Italia S.p.A. Derechos de autor
(C) 2009 Telecom Italia S.p.A.

JADE (Java Agent DEvelopment Framework) es un marco de trabajo para desarrollar sistemas multiagente que cumple con las especificaciones de la FIPA. JADE superó con éxito la 1ª prueba de interoperabilidad de la FIPA en Seúl (enero de 1999) y la 2ª prueba de interoperabilidad de la FIPA en Londres (abril de 2001).
Copyright (C) 2000 CSELT S.p.A. (C) 2001 TILab S.p.A. (C) 2002 TILab S.p.A.
Esta biblioteca es software libre; puede redistribuirla y/o modificarla según los términos de la Licencia Pública General Reducida de GNU, tal y como la ha publicado la Free Software Foundation, versión 2.1 de la licencia.
Esta biblioteca se distribuye con la esperanza de que resulte útil, pero SIN NINGÚN TIPO DE GARANTÍA; ni siquiera la garantía implícita de COMERCIABILIDAD o IDONEIDAD PARA UN FIN DETERMINADO. Consulte la Licencia Pública General Reducida de GNU para obtener más detalles.
Debería haber recibido una copia de la Licencia Pública General Reducida de GNU junto con esta biblioteca; de no ser así, escriba a la Free Software Foundation, Inc., 59 Temple Place - Suite 330, Boston, MA 02111-1307, EE. UU.

ÍNDICE

1 DESCRIPCIÓN GENERAL DE JADE 4

1.1 Contenedores y plataformas 4

1.2 AMS y DF 5

2 EL EJEMPLO DEL «INTERCAMBIO DE LIBROS» 6

3 CREACIÓN DE AGENTES JADE: LA CLASE AGENT 7

3.1 Identificadores de agentes 7

3.2 Ejecución de agentes 7

3.3 Finalización de agentes 8

3.4 Pasar argumentos a un agente 8

4 TAREAS DEL AGENTE: LA CLASE DE COMPORTAMIENTO 9

4.1 Programación y ejecución de comportamientos 9

4.2 Comportamientos puntuales, comportamientos cíclicos y comportamientos genéricos 12

4.3 Programación de operaciones en momentos determinados 13

4.4 Comportamientos necesarios en el ejemplo de la negociación de acciones 13

4.4.1 Comportamientos del agente comprador de libros 13

4.4.2 Comportamientos del agente vendedor de libros 14

5 COMUNICACIÓN ENTRE AGENTES: LA CLASE «ACLMESSAGE» 15

5.1 El lenguaje ACL 15

5.2 Envío de mensajes 16

5.3 Los mensajes de ejemplo sobre el intercambio de libros 16

5.4 Recepción de mensajes 17

5.5 Bloqueo de un comportamiento a la espera de un mensaje 17

5.6 Selección de mensajes con características determinadas de la cola de mensajes 18

5.7 Conversaciones complejas **19**

5.8 Recepción de mensajes en modo bloqueante **20**

6 EL SERVICIO DE LAS PÁGINAS AMARILLAS – LA CLASE DFSERVICE **21**

6.1 El agente DF **21**

6.2 Interacción con el DF **21**
6.2.1 Servicios de publicación 21
6.2.2 Búsqueda de servicios 22

PROGRAMACIÓN EN JADE PARA PRINCIPIANTES

Este tutorial muestra cómo crear agentes JADE sencillos y cómo hacer que ejecuten tareas y se comuniquen entre sí. JADE está escrito íntegramente en Java y los programadores de JADE trabajan en Java al desarrollar sus agentes. Por lo tanto, se da por hecho que el lector está familiarizado con el lenguaje de programación Java.

Este tutorial está estructurado de la siguiente manera. El capítulo 1 ofrece una breve visión general de JADE y presenta los conceptos de Plataforma, Contenedor, AMS y DF. El capítulo 2 presenta un ejemplo sencillo que se utilizará a lo largo de este tutorial para ilustrar los pasos necesarios para desarrollar aplicaciones sencillas basadas en agentes con JADE. El capítulo 3 se centra en la creación de agentes y explica las características básicas de las clases `Agent` y `AID`. El capítulo 4 explica cómo hacer que los agentes de JADE ejecuten tareas y presenta la clase «`Behaviour`». El capítulo 5 describe cómo hacer que los agentes se comuniquen e introduce las clases «`ACLMessage`» y «`MessageTemplate`». El capítulo 6 ilustra cómo aprovechar el servicio de Páginas Amarillas que ofrece el agente DF a través de la clase «`DFService`».

Además de las características básicas ilustradas en este tutorial, JADE ofrece una serie de características avanzadas, como la compatibilidad con protocolos de interacción complejos y el uso de ontologías definidas por el usuario. Para conocer estas características, se recomienda a los lectores consultar la Guía del programador de JADE, la Guía del administrador y los tutoriales específicos disponibles en el sitio web de JADE.

1 DESCRIPCIÓN GENERAL DE JADE

JADE es un middleware que facilita el desarrollo de sistemas multiagente. Incluye

- Un **entorno de ejecución** en el que los agentes de JADE pueden «vivir» y que debe estar activo en un host determinado antes de que se puedan ejecutar uno o más agentes en dicho host.
- Una **biblioteca** de clases que los programadores deben o pueden utilizar (directamente o especializándolas) para desarrollar sus agentes.
- Un conjunto de **herramientas gráficas** que permite administrar y supervisar la actividad de los agentes en ejecución.

1.1 Contenedores y plataformas

Cada instancia en ejecución del entorno de ejecución de JADE se denomina ***«contenedor»,*** ya que puede contener varios agentes. El conjunto de contenedores activos se denomina ***«plataforma»***. En una plataforma debe haber siempre activo un único ***contenedor principal*** especial, y todos los demás contenedores se registran en él tan pronto como se inician. De ello se deduce que el primer contenedor en iniciarse en una plataforma debe ser un contenedor principal, mientras que todos los demás contenedores deben ser contenedores «normales» (es decir, no principales) y se les debe «indicar» dónde encontrar (host y puerto) su contenedor principal (es decir, el contenedor principal en el que deben registrarse).

Si se inicia otro contenedor principal en algún punto de la red, este constituye una plataforma diferente en la que los nuevos contenedores normales pueden registrarse. La figura 1 ilustra los conceptos anteriores mediante un escenario de ejemplo que muestra dos plataformas JADE compuestas por 3 y 1 contenedores, respectivamente. Los agentes JADE se identifican mediante un nombre único y, siempre que conozcan el nombre de los demás, pueden comunicarse de forma transparente independientemente de su ubicación real: en el mismo contenedor (p. ej., los agentes A2 y A3 de la figura 1), en contenedores diferentes de la misma plataforma (p. ej., A1 y A2) o en plataformas diferentes (p. ej., A4 y A5).

Los desarrolladores no tienen que saber cómo funciona el entorno de ejecución de JADE. Solo tienen que iniciarlo antes de ejecutar sus agentes. **El inicio de JADE como contenedor principal o normal y la ejecución de agentes en él,**

se describe en el Tutorial de administración de JADE o, con más detalle, en la Guía del administrador disponible en el sitio web de JADE. No obstante, para mayor comodidad, a continuación se incluyen algunos ejemplos.

La siguiente línea de comandos inicia un contenedor principal activando la opción de la interfaz gráfica de usuario de gestión de JADE (-gui). <classpath> debe incluir todas las clases de JADE, además de todas las clases específicas de la aplicación que sean necesarias.

```
java -cp <classpath> jade.Boot -gui
```

La siguiente línea de comandos inicia un contenedor periférico (opción -container) que se registra en un contenedor principal que se ejecuta en el host avalon.tilab.com (opción -host) y activa un agente llamado john de la clase myPackage.MyClass (opción -agents)

```
java -cp <classpath> jade.Boot -container -host avalon.tilab.com -agents john:myPackage.myClass
```

1.2 AMS y DF

Además de la capacidad de aceptar registros de otros contenedores, un contenedor principal se diferencia de los contenedores normales en que alberga dos agentes especiales (que se inician automáticamente al arrancar el contenedor principal).

El **AMS** (Sistema de Gestión de Agentes), que proporciona el servicio de nombres (es decir, garantiza que cada agente de la plataforma tenga un nombre único) y representa la autoridad en la plataforma (por ejemplo, es posible crear o eliminar agentes en contenedores remotos solicitándolo al AMS). Este tutorial no explica cómo interactuar con el AMS, ya que esto forma parte de la programación avanzada de JADE.

El **DF** (Facilitador de directorios), que proporciona un servicio de «Páginas Amarillas» mediante el cual un agente puede encontrar otros agentes que presten los servicios que necesita para alcanzar sus objetivos. El uso del servicio de «Páginas Amarillas» proporcionado por el agente DF se ilustra en el capítulo 6.

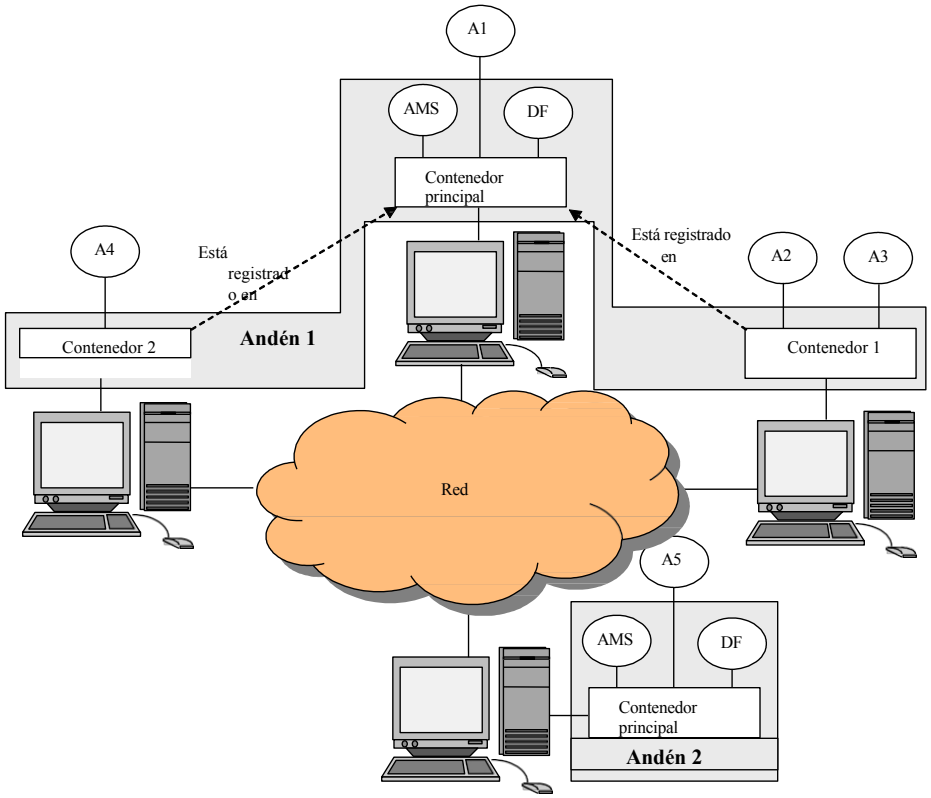


Figura 1: Contenedores y plataformas

2 EL EJEMPLO DEL «COMERCIO DE LIBROS»

En este capítulo se presenta un ejemplo sencillo que se utilizará a lo largo de este tutorial para ilustrar los pasos necesarios para desarrollar aplicaciones basadas en agentes con JADE. El escenario que se plantea en este ejemplo incluye algunos agentes que venden libros y otros que los compran en nombre de sus usuarios.

Cada agente comprador recibe el título del libro que debe comprar (el «libro objetivo») como argumento de línea de comandos y solicita periódicamente a todos los agentes vendedores conocidos que le presenten una oferta. Tan pronto como recibe una oferta, el agente comprador la acepta y emite una orden de compra. Si más de un agente vendedor presenta una oferta, el agente comprador acepta la mejor (el precio más bajo). Una vez comprado el libro objetivo, el agente comprador finaliza su actividad.

Cada agente vendedor dispone de una interfaz gráfica de usuario (GUI) mínima mediante la cual el usuario puede introducir nuevos títulos (y el precio asociado) en el catálogo local de libros en venta. Los agentes vendedores esperan continuamente solicitudes de los agentes compradores. Cuando se les pide que presenten una oferta por un libro, comprueban si el libro solicitado se encuentra en su catálogo y, en ese caso, responden indicando el precio. De lo contrario, rechazan la solicitud. Cuando reciben una orden de compra, la atienden y retiran el libro solicitado de su catálogo.

Todas las cuestiones relacionadas con el pago electrónico quedan fuera del alcance de este tutorial y no se tienen en cuenta.

Los códigos fuente completos de este ejemplo están disponibles entre los ejemplos que se proporcionan con JADE en el
paquete `examples.bookTrading`.

3 CREACIÓN DE AGENTES DE JADE: LA CLASE AGENT

Crear un agente JADE es tan sencillo como definir una clase que extienda la clase `jade.core.Agent` e implementar el método `setup()`, tal y como se muestra en el código siguiente.

```
import jade.core.Agent;

public class BookBuyerAgent extends Agent { protected
  void setup() {
    // Imprime un mensaje de bienvenida
    System.out.println(«¡Hola! El agente comprador «+getAID().getName()+» está listo.»);
  }
}
```

El método `setup()` está pensado para incluir las inicializaciones del agente. La tarea real que debe realizar un agente
se lleva a cabo normalmente dentro de los «comportamientos», tal y como se describirá en el capítulo 4.

3.1 Identificadores de agentes

Cada agente se identifica mediante un «identificador de agente», representado como una instancia de la clase `jade.core.AID`. El método `getAID()` de la clase `Agent` permite recuperar el identificador del agente. Un objeto `AID` incluye un nombre único a nivel global, además de varias direcciones. En JADE, el nombre tiene el formato *<apodo>@<nombre-de-la-plataforma>,* de modo que un agente llamado *Peter* que resida en una plataforma llamada *P1* tendrá *Peter@P1* como nombre único a nivel global. Las direcciones incluidas en el AID son las direcciones de la plataforma en la que reside el agente. Estas direcciones solo se utilizan cuando un agente necesita comunicarse con otro agente que resida en una plataforma diferente. Los desarrolladores solo deben tenerlas en cuenta en casos concretos que quedan fuera del alcance de este tutorial.

Si se conoce el apodo de un agente, su AID se puede obtener de la siguiente manera:

```
String apodo = «Peter»;
AID id = new AID(nickname, AID.ISLOCALNAME);
```

La constante `ISLOCALNAME` indica que el primer parámetro representa el apodo (local a la plataforma) y no el nombre globalmente único del agente.

3.2 Ejecución de agentes

El agente creado se puede compilar de la siguiente manera:

```
javac -classpath <clases-JADE> BookBuyerAgent.java
```

Para ejecutar el agente compilado, es necesario iniciar el entorno de ejecución de JADE y elegir un apodo para el agente que se va a ejecutar:

```
java -classpath <clases-JADE>;. jade.Boot buyer:BookBuyerAgent
```

Se puede encontrar más información sobre la compilación y ejecución de agentes en el Tutorial de administración de JADE o en la Guía del administrador de JADE, disponibles en la página web de JADE. El resultado del comando introducido es el siguiente.

```
C:\jade>java -classpath <JADE-classes>;. jade.Boot buyer:BookBuyerAgent 5
de mayo de 2008 11:06:45 jade.core.Runtime beginContainer
INFO: -------------------------------------------------
    Esta es una instantánea de JADE —revisión 5995 del 3 de septiembre de
    2007 a las 09:45:22— descargada como código abierto, bajo las
    restricciones de la licencia LGPL,
    en http://jade.tilab.com/
-----------------------------------------------------------
5 de mayo de 2008 11:06:51 jade.core.BaseService init
```

```
INFO: Servicio jade.core.management.AgentManagement inicializado
5 de mayo de 2008 11:06:51 jade.core.BaseService init
INFO: Servicio jade.core.messaging.Messaging inicializado
5 de mayo de 2008 11:06:52 jade.core.BaseService init
INFO: Servicio jade.core.mobility.AgentMobility inicializado
el 5 de mayo de 2008 a las 11:06:52 jade.core.BaseService
init
INFO: Servicio jade.core.event.Notification inicializado
5 de mayo de 2008 a las 11:06:52 jade.core.messaging.MessagingService
clearCachedSlice INFO: Borrando la caché
5 de mayo de 2008 11:06:53 jade.mtp.http.HTTPServer <init>
INFO: HTTP-MTP utilizando el analizador XML
com.sun.org.apache.xerces.internal.parsers.SAXParser 5 de mayo de 2008 11:06:54
jade.core.messaging.MessagingService boot
INFO: Direcciones MTP:
http://NBNT2004130496.telecomitalia.local:7778/acc
5 de mayo de 2008 11:06:54 jade.core.AgentContainerImpl
joinPlatform INFORMACIÓN:
El contenedor de agentes Main-Container@NBNT2004130496 está listo.
------------------------------------------------------------
¡Hola! El agente comprador buyer@NBNT2004130496:1099/JADE está listo.
```

La primera parte de la salida anterior es el aviso legal de JADE que se muestra cada vez que se inicia el entorno de ejecución de JADE. A continuación, se produce la inicialización de los servicios del núcleo (los servicios del núcleo son elementos internos de JADE y no se describen en este documento) activados en la plataforma iniciada. Por último, la indicación de que un contenedor denominado «Main-Container» está listo completa el inicio del entorno de ejecución de JADE. Cuando el entorno de ejecución de JADE está en funcionamiento, se inicia nuestro agente y muestra su mensaje de bienvenida. El apodo del agente es «buyer», tal y como especificamos en la línea de comandos. El nombre de la plataforma «NBNT2004130496:1099/JADE» se asigna automáticamente en función del host y el puerto en los que se está ejecutando JADE (consulta la Guía del administrador de JADE para asignar un nombre a la plataforma).

3.3 Finalización del agente

Aunque no tenga nada más que hacer tras mostrar el mensaje de bienvenida, nuestro agente sigue en ejecución. Para que finalice, es necesario llamar a su método `doDelete()`. De forma similar al método `setup()`, que es invocado por el tiempo de ejecución de JADE tan pronto como se inicia un agente y está destinado a incluir las inicializaciones del agente, el método `takeDown()` se invoca justo antes de que un agente finalice y está destinado a incluir las operaciones de limpieza del agente.

3.4 Pasar argumentos a un agente

Los agentes pueden recibir argumentos de inicio especificados en la línea de comandos. Estos argumentos se pueden recuperar, como una matriz de `objetos`, mediante el método `getArguments()` de la clase `Agent`. Como se mencionó en el capítulo 2, queremos que nuestro `BookBuyerAgent` reciba el título del libro que se va a comprar como argumento de la línea de comandos. Para lograrlo, lo modificamos de la siguiente manera[1] .

```
import jade.core.Agent; import
jade.core.AID;

public class BookBuyerAgent extends Agent {
  // El título del libro que se va a
  comprar private String targetBookTitle;
  // La lista de agentes vendedores conocidos
  private AID[] sellerAgents = {new AID(«seller1», AID.ISLOCALNAME),
                                new AID("seller2", AID.ISLOCALNAME)};
```

[1] Ten en cuenta que la lista de agentes vendedores conocidos a los que enviar solicitudes es fija. En el capítulo 6 describiremos cómo detectarlos dinámicamente.

```
  // Coloca aquí las inicializaciones de
  los agentes protected void setup() {
    // Imprime un mensaje de bienvenida
    System.out.println(«¡Hola! El agente comprador «+getAID().getName()+» está listo.»);

    // Obtener el título del libro que se va a comprar como
    argumento de inicio Object[] args = getArguments();
    if (args != null && args.length > 0) {
      targetBookTitle = (String) args[0];
      System.out.println(«Intentando comprar « + targetBookTitle);
    }
    else {
      // Hacer que el agente termine inmediatamente
      System.out.println("No se ha especificado ningún título de
      libro"); doDelete();
    }
  }

  // Coloca aquí las operaciones de
  limpieza del agente protected void
  takeDown() {
    // Imprimir un mensaje de cierre
    System.out.println("El agente comprador "+getAID().getName()+" se está cerrando.");
  }
}
```

Los argumentos de la línea de comandos se especifican entre paréntesis y separados por espacios [2].

```
C:\jade>java jade.Boot buyer:BookBuyerAgent(El-Señor-de-los-Anillos)
...
...
5 de mayo de 2008 11:11:00 jade.core.AgentContainerImpl
joinPlatform INFO:----------------------------------------
El contenedor de agentes Main-Container@NBNT2004130496 está listo.
------------------------------------------------------------
¡Hola! El agente comprador buyer@NBNT2004130496:1099/JADE está
listo. Intentando comprar El-Señor-de-los-Anillos
```

4 TAREAS DEL AGENTE – LA CLASE DE COMPORTAMIENTO

Como se mencionó en el capítulo 3, la tarea real que debe realizar un agente se lleva a cabo normalmente mediante «comportamientos». Un comportamiento representa una tarea que un agente puede realizar y se implementa como un objeto de una clase que extiende `jade.core.behaviours.Behaviour`. Para que un agente ejecute la tarea implementada por un objeto de comportamiento, basta con añadir el comportamiento al agente mediante el método `addBehaviour()` de la clase `Agent`. Los comportamientos se pueden añadir en cualquier momento: al iniciar un agente (en el método `setup()`) o desde el interior de otros comportamientos.

Cada clase que extienda `Behaviour` debe implementar el método `action()`, que define las operaciones que se deben realizar cuando el comportamiento está en ejecución, y el método `done()` (que devuelve un valor `booleano`), que especifica si un comportamiento ha finalizado o no y si debe eliminarse del conjunto de comportamientos que está llevando a cabo un agente.

4.1 Programación y ejecución de comportamientos

Un agente puede ejecutar varios comportamientos simultáneamente. Sin embargo, es importante señalar que la programación de los comportamientos en un agente no es preemptiva (como en el caso de los hilos de Java), sino cooperativa. **Esto significa que cuando un**

[2] Al utilizar JADE con el complemento LEAP, los argumentos se separan mediante dos puntos («;») en lugar de espacios.

comportamiento se programa para su ejecución, se invoca su método `action()` y este **se ejecuta hasta que devuelve un resultado.** Por lo tanto, es el programador quien define cuándo un agente pasa de la ejecución de un comportamiento a la ejecución del siguiente.

Aunque supone un pequeño esfuerzo adicional para los programadores, este enfoque presenta varias ventajas.

- Permite tener un único hilo de Java por agente (lo cual es muy importante, especialmente en entornos con recursos limitados, como los teléfonos móviles).

- Ofrece un mejor rendimiento, ya que el cambio de comportamiento es mucho más rápido que el cambio de hilo en Java.

- Elimina todos los problemas de sincronización entre comportamientos simultáneos que acceden a los mismos recursos (lo que además mejora el rendimiento), ya que todos los comportamientos se ejecutan en el mismo hilo de Java.

- Cuando se produce un cambio de comportamiento, el estado de un agente no incluye ninguna información de pila, por lo que es posible tomar una «instantánea» del mismo. Esto permite implementar importantes funciones avanzadas, como guardar el estado de un agente en un almacenamiento persistente para reanudarlo más tarde (persistencia del agente) o transferirlo a otro contenedor para su ejecución remota (movilidad del agente). Sin embargo, la persistencia y la movilidad son funciones avanzadas de JADE y quedan fuera del alcance de este tutorial.

La ruta de ejecución del hilo del agente [3] se muestra en la figura 2.

[3] En JADE hay un único hilo de Java por agente. No obstante, dado que los agentes de JADE están escritos en Java, los programadores pueden iniciar nuevos hilos de Java en cualquier momento si lo necesitan. Si lo haces, recuerda prestar atención, ya que las ventajas mencionadas en esta sección dejarán de ser válidas.

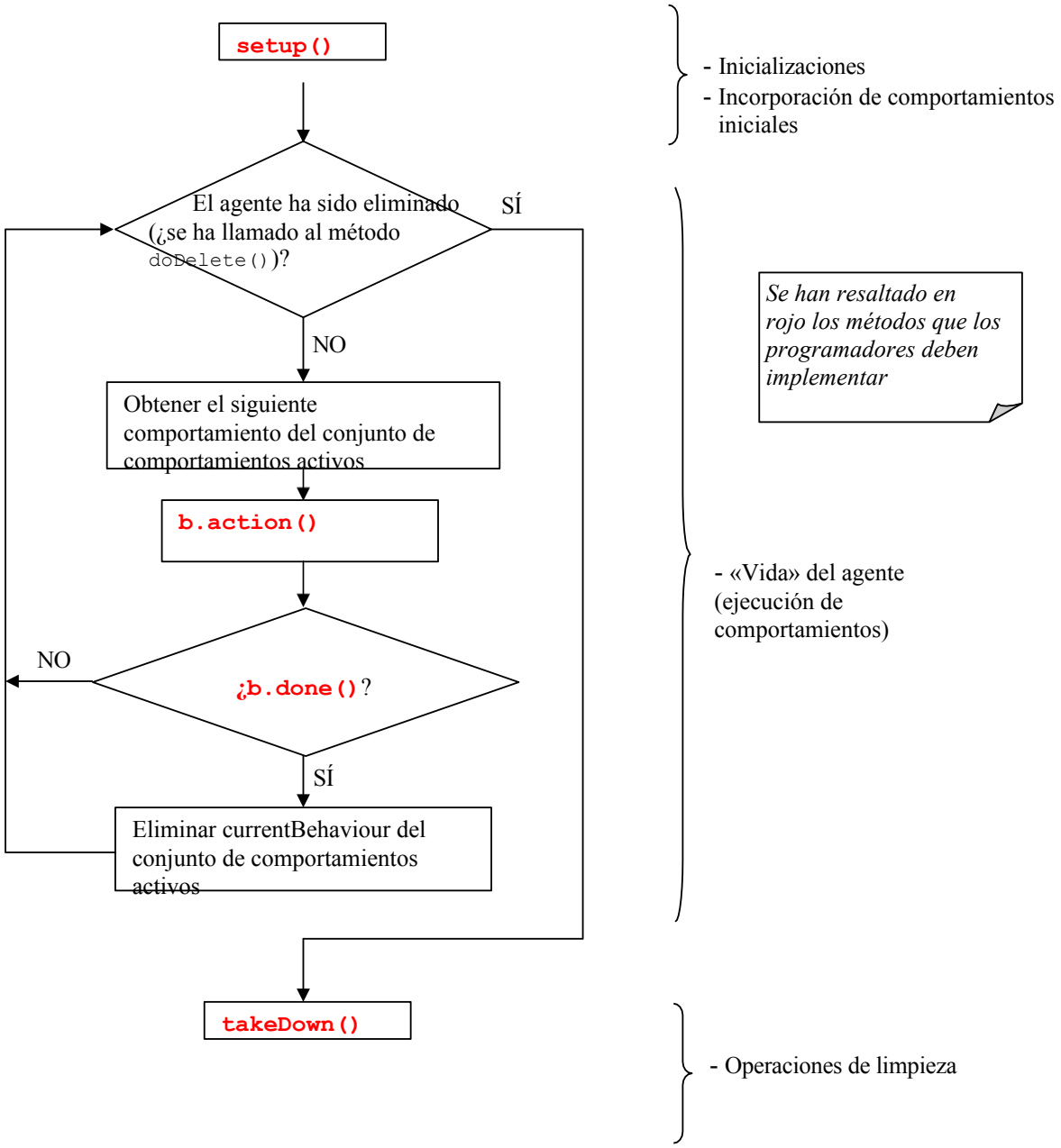


Figura 2. Ruta de ejecución del hilo del agente

Teniendo en cuenta el mecanismo de programación descrito, es importante destacar que un comportamiento como el que se muestra a continuación impide que se ejecute cualquier otro comportamiento, ya que su método `action()` nunca devuelve ningún valor.

```
public class OverbearingBehaviour extends Behaviour {
  public void action() {
    while (true) {
      // hacer algo
    }
  }

  público booleano done() {
    devuelve true;
  }
```

```
}
```

Cuando no hay comportamientos disponibles para su ejecución, el hilo del agente entra en estado de reposo para no consumir tiempo de CPU. Se reactiva tan pronto como vuelve a haber un comportamiento disponible para su ejecución.

4.2 Comportamientos puntuales, comportamientos cíclicos y comportamientos genéricos

Podemos distinguir entre tres tipos de comportamientos.

1) Los comportamientos «de una sola vez», que se completan inmediatamente y cuyo método `action()` se ejecuta solo una vez. La clase `jade.core.behaviours.OneShotBehaviour` ya implementa el método `done()` devolviendo `true` y puede ampliarse fácilmente para implementar comportamientos de una sola vez.

```
public class MyOneShotBehaviour extends OneShotBehaviour { public
  void action() {
    // realizar la operación X
  }
}
```

La operación X se realiza solo una vez.

2) Comportamientos «cíclicos» que nunca finalizan y cuyo método `action()` ejecuta las mismas operaciones cada vez que se invoca. La clase `jade.core.behaviours.CyclicBehaviour` ya implementa el método `done()` devolviendo `false` y puede ampliarse fácilmente para implementar comportamientos cíclicos.

```
public class MyCyclicBehaviour extends CyclicBehaviour {
  public void action() {
    // realizar la operación Y
  }
}
```

La operación Y se realiza de forma repetitiva indefinidamente (hasta que el agente que lleva a cabo el comportamiento anterior finaliza).

3) Comportamientos genéricos que incorporan un estado y ejecutan diferentes operaciones en función de dicho estado. Se completan cuando se cumple una condición determinada.

```
public class MyThreeStepBehaviour extends Behaviour {
  private int step = 0;
  public void action() { switch
     (step) {
    case 0:
      // realizar la
      operación X step++;
      break;
    case 1:
      // realizar la
      operación Y step++;
      break;
    case 2:
      // realizar la
      operación Z step++;
      break;
    }
  }

  public boolean done() {
    return step == 3;
  }
}
```

Las operaciones X, Y y Z se realizan una tras otra y, a continuación, el comportamiento finaliza.

JADE ofrece la posibilidad de combinar comportamientos simples para crear comportamientos complejos. Sin embargo, esta característica queda fuera del alcance de este documento. Consulte la documentación de Javadoc de `SequentialBehaviour`, `ParallelBehaviour` y `FSMBehaviour` para obtener más detalles.

4.3 Programación de operaciones en momentos determinados

JADE proporciona dos clases ya preparadas (en el paquete `jade.core.behaviours`) mediante las cuales es posible implementar fácilmente comportamientos que ejecutan determinadas operaciones en momentos concretos.

1) El `comportamiento` `WakerBehaviour`, cuyos métodos `action()` y `done()` ya están implementados de tal forma que ejecutan el método abstracto `handleElapsedTimeout()` una vez que expira un tiempo de espera determinado (especificado en el constructor). Tras la ejecución del método `handleElapsedTimeout()`, el comportamiento finaliza.

```
public class MyAgent extends Agent { protected
  void setup() {
    System.out.println(«Añadiendo el comportamiento
    waker»); addBehaviour(new WakerBehaviour(this,
    10000) {
      protected void handleElapsedTimeout() {
        // realizar la operación X
      }
    } );
  }
}
```

La operación X se realiza 10 segundos después de que aparezca el mensaje «Adding waker behaviour».

2) El `TickerBehaviour`, cuyos métodos `action()` y `done()` ya están implementados de tal forma que ejecutan el método abstracto `onTick()` de forma repetitiva, esperando un periodo determinado (especificado en el constructor) tras cada ejecución. Un `TickerBehaviour` nunca finaliza.

```
public class MyAgent extends Agent { protected
  void setup() {
    addBehaviour(new TickerBehaviour(this, 10000)) {
      protected void onTick() {
        // realizar la operación Y
      }
    } );
  }
}
```

La operación Y se realiza periódicamente cada 10 segundos.

4.4 Comportamientos necesarios en el ejemplo del comercio de libros

Una vez descritos los tipos básicos de comportamiento, pasemos ahora a analizar qué comportamientos deben llevar a cabo el agente comprador de libros y el agente vendedor de libros de nuestro ejemplo de comercio de libros.

4.4.1 Comportamientos del agente comprador de libros

Tal y como se describe en el capítulo 2, un agente «Book-buyer» solicita periódicamente a los agentes vendedores el libro que se le ha encargado comprar. Podemos lograrlo fácilmente utilizando un `TickerBehaviour` que, en cada tick, añada otro comportamiento que se encargue efectivamente de la solicitud a los agentes vendedores. A continuación se muestra cómo se puede modificar el método `setup()` de nuestra clase `BookBuyerAgent`.

```
protected void setup() {
  // Imprime un mensaje de bienvenida
  System.out.println("¡Hola! El agente comprador "+getAID().getName()+" está listo.");

  // Obtiene el título del libro que se va a comprar como
  argumento de inicio Object[] args = getArguments();
  if (args != null && args.length > 0) { targetBookTitle
    = (String) args[0];
    System.out.println(«Intentando comprar « + targetBookTitle);

    // Añadir un TickerBehaviour que programe una solicitud a los agentes vendedores cada
    minuto
```

```
    addBehaviour(new TickerBehaviour(this, 60000) {
      protected void onTick() {
        myAgent.addBehaviour(new RequestPerformer());
      }
    } );
  }
  else {
    // Hacer que el agente termine
    System.out.println(«No se ha especificado ningún título de
    libro de destino»); doDelete();
  }
}
```

Fíjate en el uso de la variable protegida `myAgent`: cada comportamiento tiene un puntero al agente que se está lo está ejecutando.

El comportamiento `RequestPerformer, que` se encarga realmente de la solicitud a los agentes vendedores, se describirá en el capítulo 5, donde trataremos la comunicación entre agentes.

4.4.2 Comportamientos de los agentes vendedores de libros

Tal y como se describe en el capítulo 2, cada agente «Librero» espera las solicitudes de los agentes «Comprador» y las atiende. Estas solicitudes pueden consistir en peticiones para proporcionar una oferta por un libro o en órdenes de compra. Un posible diseño para lograrlo consiste en hacer que un agente «Librero» ejecute dos comportamientos cíclicos: uno dedicado a atender las solicitudes de oferta y otro dedicado a atender las órdenes de compra. En el capítulo 5, donde trataremos la comunicación entre agentes, se describe cómo se detectan y atienden realmente las solicitudes entrantes de los agentes compradores. Además, debemos hacer que el agente «Book-seller» ejecute un comportamiento puntual que actualice el catálogo de libros disponibles para la venta cada vez que el usuario añada un nuevo libro desde la interfaz gráfica de usuario. A continuación se muestra cómo se puede implementar la clase «`BookSellerAgent`» (las clases «`OfferRequestsServer`» y «`PurchaseOrdersServer`» se presentarán en el capítulo 5).

```
import jade.core.Agent;
import jade.core.behaviours.*; import

java.util.*;

public class BookSellerAgent extends Agent {
  // El catálogo de libros a la venta (asigna el título de un libro a su precio)
  private Hashtable catalogue;
  // La interfaz gráfica de usuario mediante la cual el usuario puede añadir
  libros al catálogo private BookSellerGui myGui;

  // Coloca aquí las inicializaciones del
  agente protected void setup() {
    // Crear el catálogo catalogue =
    new Hashtable();

    // Crear y mostrar la interfaz gráfica
    myGui = new BookSellerGui(this);
    myGui.show();

    // Añadir el comportamiento que atiende las solicitudes de oferta de los
    agentes compradores addBehaviour(new OfferRequestsServer());

    // Añadir el comportamiento que gestiona los pedidos de compra de los
    agentes compradores addBehaviour(new PurchaseOrdersServer());
  }

  // Coloca aquí las operaciones de
  limpieza del agente protected void
  takeDown() {
```

```
    // Cerrar la interfaz
    gráfica de usuario
    myGui.dispose();
    // Mostrar un mensaje de cierre
    System.out.println("Agente vendedor "+getAID().getName()+" finalizando.");
  }

  /**
     La interfaz gráfica de usuario invoca este método cuando el usuario añade un nuevo libro a
     la venta
   */
  public void updateCatalogue(final String title, final int price) { addBehaviour(new
    OneShotBehaviour() {
      public void action() {
        catalogue.put(title, new Integer(price));
      }
    } );
  }
}
```

La clase `BookSellerGui` es una sencilla interfaz gráfica de usuario (GUI) de Swing y no se presenta aquí, ya que queda fuera del alcance de este tutorial. Su código está disponible entre los archivos fuente incluidos con este tutorial.

5 COMUNICACIÓN ENTRE AGENTES: LA CLASE ACLMESSAGE

Una de las características más importantes que ofrecen los agentes JADE es la capacidad de comunicarse. El paradigma de comunicación adoptado es el **paso asíncrono de mensajes**. Cada agente dispone de una especie de buzón (la cola de mensajes del agente) en el que el entorno de ejecución de JADE envía los mensajes remitidos por otros agentes. Cada vez que se envía un mensaje a la cola de mensajes, se notifica al agente receptor. Sin embargo, el momento en que el agente recoge realmente el mensaje de la cola para procesarlo depende totalmente del programador.

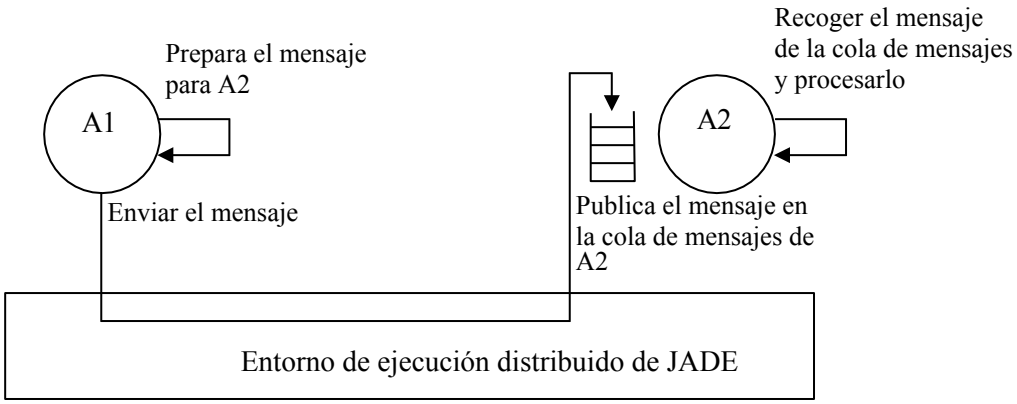


Figura 3. El paradigma de paso de mensajes asíncrono de JADE

5.1 El lenguaje ACL

Los mensajes intercambiados por los agentes JADE tienen un formato especificado por el lenguaje ACL, definido por la norma internacional FIPA (http://www.fipa.org) para la interoperabilidad de agentes. Este formato consta de varios campos y, en particular:

- El ***remitente*** del mensaje
- La lista de ***destinatarios***
- La intención comunicativa (también denominada «performativa»), que indica lo que el remitente pretende

con el envío del mensaje. La intención performativa puede ser REQUEST, si el remitente desea que el destinatario realice una acción; INFORM, si el remitente desea que el destinatario tenga conocimiento de un hecho; QUERY_IF, si el remitente desea saber si se cumple o no una condición determinada; CFP (solicitud de propuesta); PROPOSE, ACCEPT_PROPOSAL y REJECT_PROPOSAL, si el remitente y el destinatario están inmersos en una negociación; y otras más.

- El ***contenido***, es decir, la información real incluida en el mensaje (por ejemplo, la acción que debe realizarse en un mensaje de SOLICITUD, el hecho que el remitente desea comunicar en un mensaje de INFORMACIÓN…).

- ***El lenguaje*** del contenido, es decir, la sintaxis utilizada para expresar el contenido (tanto el remitente como el destinatario deben ser capaces de codificar/analizar expresiones que cumplan con esta sintaxis para que la comunicación sea efectiva).

- La ***ontología***, es decir, el vocabulario de los símbolos utilizados en el contenido y su significado (tanto el remitente como el destinatario deben atribuir el mismo significado a los símbolos para que la comunicación sea efectiva).

- Algunos campos utilizados para gestionar varias conversaciones simultáneas y para especificar plazos de espera para recibir una respuesta, como ***conversation-id***, ***reply-with***, ***in-reply-to*** y ***reply-by***.

Un mensaje en JADE se implementa como un objeto de la clase `jade.lang.acl.ACLMessage`, que proporciona métodos `get` y `set` para gestionar todos los campos de un mensaje.

5.2 Envío de mensajes

Enviar un mensaje a otro agente es tan sencillo como rellenar los campos de un objeto `ACLMessage` y, a continuación, llamar al método `send()` de la clase `Agent`. El código siguiente informa a un agente cuyo apodo es *Peter* de que *hoy llueve*.

```
ACLMessage msg = new ACLMessage(ACLMessage.INFORM);
msg.addReceiver(new AID(«Peter», AID.ISLOCALNAME));
msg.setLanguage(«English»); msg.setOntology(«Weather-
forecast-ontology»); msg.setContent(«Today it’s
raining»);
send(msg);
```

5.3 Los mensajes del ejemplo de intercambio de libros

Teniendo en cuenta nuestro ejemplo de compraventa de libros, podemos utilizar convenientemente el performativo `CFP` (convocatoria de propuestas) para los mensajes que los agentes compradores envían a los agentes vendedores con el fin de solicitar una oferta por un libro. El performativo `PROPOSE` puede utilizarse para los mensajes que contienen ofertas de los vendedores, y el performativo `ACCEPT_PROPOSAL` para los mensajes que contienen la aceptación de la oferta, es decir, los pedidos de compra. Por último, el performativo `REFUSE` se utilizará para los mensajes enviados por los agentes vendedores cuando el libro solicitado no se encuentre en su catálogo. En ambos tipos de mensajes enviados por los agentes compradores, suponemos que el contenido del mensaje es el título del libro. El contenido de los mensajes `PROPOSE` será el precio del libro. A modo de ejemplo, a continuación se muestra cómo se puede crear y enviar un mensaje `CFP`.

```
// Mensaje que contiene una solicitud de oferta
ACLMessage cfp = new ACLMessage(ACLMessage.CFP);
for (int i = 0; i < sellerAgents.length; ++i) {
  cfp.addReceiver(sellerAgents[i]);
}
cfp.setContent(targetBookTitle);
myAgent.send(cfp);
```

5.4 Recepción de mensajes

Como se ha mencionado anteriormente, el entorno de ejecución de JADE envía automáticamente los mensajes a la cola de mensajes privada del receptor tan pronto como llegan. Un agente puede recoger mensajes de su cola de mensajes mediante el método `receive()`. Este método devuelve el primer mensaje de la cola de mensajes (eliminándolo) o `null` si la cola de mensajes está vacía, y retorna inmediatamente.

```
ACLMessage msg = receive(); if
(msg != null) {
  // Procesar el mensaje
}
```

5.5 Bloqueo de un comportamiento a la espera de un mensaje

Muy a menudo, los programadores necesitan implementar comportamientos que procesen mensajes recibidos por otros agentes. Este es el caso de los comportamientos `OfferRequestsServer` y `PurchaseOrdersServer` a los que se hace referencia en 4.4.2, donde debemos atender los mensajes de los agentes compradores que contienen solicitudes de ofertas y órdenes de compra.

Dichos comportamientos deben ejecutarse de forma continua (comportamientos cíclicos) y, en cada ejecución de su método `action()`, deben comprobar si se ha recibido un mensaje y procesarlo. Los dos comportamientos son muy similares. A continuación presentamos el comportamiento `OfferRequestsServer`. Consulte los códigos fuente disponibles entre los ejemplos de JADE para ver el código de `PurchaseOrdersServer`.

```
/**
   Clase interna OfferRequestsServer.
   Este es el comportamiento que utilizan los agentes vendedores de libros para
   atender las solicitudes de oferta entrantes de los agentes compradores.
   Si el libro solicitado se encuentra en el catálogo local, el agente vendedor
   responde con un mensaje PROPOSE en el que se especifica el precio. De lo
   contrario, se envía un mensaje REFUSE.
 */
clase privada OfferRequestsServer extiende CyclicBehaviour { public
  void action() {
    ACLMessage msg = myAgent.receive(); if
    (msg != null) {
      // Mensaje recibido. Procesarlo String
      title = msg.getContent(); ACLMessage
      reply = msg.createReply();

      Integer price = (Integer) catalogue.get(title);
      if (price != null) {
        // El libro solicitado está disponible para la venta. Responder con el precio
        reply.setPerformative(ACLMessage.PROPOSE);
        reply.setContent(String.valueOf(price.intValue()));
      }
      else {
        // El libro solicitado NO está disponible para la
        venta. reply.setPerformative(ACLMessage.REFUSE);
        reply.setContent("no-disponible");
      }
      myAgent.send(reply);
    }
  }
} // Fin de la clase interna OfferRequestsServer
```

Hemos decidido implementar el comportamiento `de OfferRequestsServer` como una clase interna de la clase `BookSellerAgent`. Esto simplifica las cosas, ya que podemos acceder directamente al catálogo de libros en venta; sin embargo, no es obligatorio.

El método `createReply()` de la clase `ACLMessage` crea automáticamente un nuevo objeto `ACLMessage` configurando correctamente los destinatarios y todos los campos utilizados para controlar la conversación (conversation-id, reply-with,

in-reply-to), si los hay.

Sin embargo, si observamos la figura 2, podemos apreciar que, al añadir el comportamiento anterior, el hilo del agente inicia un bucle continuo que consume muchísima CPU. Para evitarlo, nos gustaría ejecutar el método `action()` del comportamiento `OfferRequestsServer` solo cuando se reciba un nuevo mensaje. Para ello, podemos utilizar el método `block()` de la clase `Behaviour`. Este método marca el comportamiento como «bloqueado», de modo que el agente ya no lo programa para su ejecución. Cuando se inserta un nuevo mensaje en la cola de mensajes del agente, todos los comportamientos bloqueados vuelven a estar disponibles para su ejecución, de modo que tienen la oportunidad de procesar el mensaje recibido. Por lo tanto, el método `action()` debe modificarse de la siguiente manera.

```
public void action() {
  ACLMessage msg = myAgent.receive(); if
  (msg != null) {
    // Mensaje recibido. Procesarlo
    ...
  }
  else {
    block();
  }
}
```

El código anterior es el patrón típico (y muy recomendado) para recibir mensajes dentro de un comportamiento.

5.6 Selección de mensajes con características determinadas de la cola de mensajes

Teniendo en cuenta que tanto el comportamiento `OfferRequestsServer` como el `PurchaseOrdersServer` son comportamientos cíclicos cuyo método `action()` comienza con una llamada a `myAgent.receive()`, es posible que hayas detectado un problema: ¿cómo podemos asegurarnos de que el comportamiento `OfferRequestsServer` recoja de la cola de mensajes del agente únicamente los mensajes que contengan solicitudes de oferta y que el comportamiento `PurchaseOrdersServer` recoja únicamente los mensajes que contengan órdenes de compra? Para resolver este problema, debemos modificar el código que hemos presentado hasta ahora especificando las «plantillas» adecuadas al llamar al método `receive()`. Cuando se especifica una plantilla, el método `` `receive()` `` devuelve el primer mensaje (si lo hay) que coincida con ella, mientras que ignora todos los mensajes que no coincidan. Dichas plantillas se implementan como instancias de la clase `` `jade.lang.acl.MessageTemplate` ``, que proporciona una serie de métodos de fábrica para crear plantillas de una forma muy sencilla y flexible.

Como se mencionó en el apartado 5.3, utilizamos el performativo `CFP` para los mensajes que contienen solicitudes de oferta y el performativo `ACCEPT_PROPOSAL` para los mensajes que contienen aceptaciones de propuestas, es decir, las órdenes de compra. Por lo tanto, modificamos el método `action()` del `OfferRequestsServer` para que la llamada a `myAgent.receive()` ignore todos los mensajes excepto aquellos cuyo performativo sea `CFP`.

```
public void action() {
  MessageTemplate mt = MessageTemplate.MatchPerformative(ACLMessage.CFP);
  ACLMessage msg = myAgent.receive(mt);
  if (msg != null) {
    // Se ha recibido un mensaje CFP. Procesarlo
    ...
  }
  else {
    block();
  }
}
```

5.7 Conversaciones complejas

El comportamiento `RequestPerformer` mencionado en el apartado 4.4.1 representa un ejemplo de comportamiento que lleva a cabo una conversación «compleja». Una conversación es una secuencia de mensajes intercambiados por dos o más agentes con relaciones causales y temporales bien definidas. El comportamiento `RequestPerformer` debe enviar un mensaje `CFP` a varios agentes (los agentes vendedores conocidos), recibir todas las respuestas y, en caso de que se reciba al menos una respuesta `PROPOSE`, enviar un mensaje `ACCEPT_PROPOSAL` adicional (al agente vendedor que realizó la propuesta) y recibir la respuesta. Siempre que haya que llevar a cabo una conversación, es recomendable especificar los campos de control de la conversación en los mensajes intercambiados durante la misma. Esto permite crear de forma sencilla y sin ambigüedades plantillas que se ajusten a las posibles respuestas.

```
/**
   Clase interna RequestPerformer.
   Este es el comportamiento que utilizan los agentes compradores de libros
   para solicitar a los agentes vendedores el libro en cuestión.
 */
clase privada RequestPerformer extiende Behaviour {
  private AID bestSeller; // El agente que ofrece la mejor oferta private
  int bestPrice; // El mejor precio ofrecido
  private int repliesCnt = 0; // El contador de respuestas de los agentes
  vendedores private MessageTemplate mt; // La plantilla para recibir respuestas
  private int paso = 0;

  public void acción() { switch
    (paso) {
    case 0:
      // Enviar la solicitud de oferta (CFP) a todos los vendedores
      ACLMessage cfp = new ACLMessage(ACLMessage.CFP);
      for (int i = 0; i < sellerAgents.length; ++i) {
        cfp.addReceiver(sellerAgents[i]);
      }
      cfp.setContent(targetBookTitle);
      cfp.setConversationId("book-trade");
      cfp.setReplyWith(«cfp» + System.currentTimeMillis()); // Valor único myAgent.send(cfp);
      // Preparar la plantilla para recibir propuestas
      mt = MessageTemplate.and(MessageTemplate.MatchConversationId(«book-trade»),
                               MessageTemplate.MatchInReplyTo(cfp.getReplyWith()));
      step = 1;
      break;
    case 1:
      // Recibir todas las propuestas/rechazos de los agentes
      vendedores ACLMessage reply = myAgent.receive(mt);
      if (respuesta != null) {
        // Respuesta recibida
        if (respuesta.getPerformative() == ACLMessage.PROPOSE) {
          // Se trata de una oferta
          int precio =
          Integer.parseInt(respuesta.getContent()); if
          (mejorVendedor == null || precio < mejorPrecio) {
            // Esta es la mejor oferta en este
            momento bestPrice = price;
            mejorVendedor = respuesta.getSender();
          }
        }
        repliesCnt++;
        if (repliesCnt >= sellerAgents.length) {
          // Hemos recibido todas las
          respuestas step = 2;
        }
      }
      else {
```

```
        block();
      }
      break;
    caso 2:
      // Enviar la orden de compra al vendedor que ha presentado la mejor oferta
      ACLMessage pedido = new ACLMessage(ACLMessage.ACCEPT_PROPOSAL);
      pedido.addReceiver(mejorVendedor);
      order.setContent(targetBookTitle);
      order.setConversationId("book-trade");
      order.setReplyWith("order"+System.currentTimeMillis());
      myAgent.send(order);
      // Preparar la plantilla para recibir la respuesta a la orden de compra
      mt = MessageTemplate.and(MessageTemplate.MatchConversationId(«book-trade»),
                               MessageTemplate.MatchInReplyTo(order.getReplyWith()));
      step = 3;
      break;
    case 3:
      // Recibir la respuesta a la orden de
      compra reply = myAgent.receive(mt);
      if (reply != null) {
        // Se ha recibido la respuesta a la orden de compra
        if (respuesta.getPerformative() == ACLMessage.INFORM) {
          // Compra realizada con éxito. Podemos finalizar
          System.out.println(targetBookTitle + «comprado con éxito.»);
          System.out.println(«Precio = » + bestPrice);
          myAgent.doDelete();
        }
        step = 4;
      }
      else {
        block();
      }
      break;
    }
  }

  public boolean done() {
    return ((step == 2 && bestSeller == null) || step == 4);
  }
} // Fin de la clase interna RequestPerformer
```

Las conversaciones complejas suelen llevarse a cabo siguiendo un protocolo de interacción bien definido. JADE ofrece un amplio soporte para implementar conversaciones que sigan protocolos de interacción en el paquete `jade.proto`. En concreto, la conversación que hemos implementado sigue un protocolo «Contract-net» y se podría haber implementado muy fácilmente utilizando la clase `jade.proto.ContractNetInitiator`. No obstante, el soporte para los protocolos de interacción queda fuera del alcance de este tutorial. Consulta la Guía del programador de JADE y la documentación de Javadoc para obtener más detalles.

5.8 Recepción de mensajes en modo bloqueante

Además del método `receive()`, la clase `Agent` también proporciona el método `blockingReceive()` que, como su nombre indica, es una llamada bloqueante: no devuelve ningún valor hasta que haya un mensaje en la cola de mensajes del agente. También está disponible la versión sobrecargada que toma un `MessageTemplate` como parámetro (no devuelve ningún valor hasta que haya un mensaje que coincida con la plantilla especificada).

Es importante destacar que el método `blockingReceive()` bloquea efectivamente el hilo del agente. Por lo tanto, si se llama a `` `blockingReceive()` `` desde dentro de un comportamiento, esto impide que se ejecuten todos los demás comportamientos hasta que la llamada a `` `blockingReceive()` `` devuelva un resultado. Teniendo en cuenta lo anterior, una buena práctica de programación para recibir mensajes es: utilizar `` `blockingReceive()` `` en los métodos `` `setup()` `` y `` `takeDown()` ``; utilizar `` `receive()` `` en combinación con `` `Behaviour.block()` `` (como se muestra en el apartado 5.5) dentro de los comportamientos.

En el código que hemos escrito hasta ahora hemos supuesto que existe un conjunto fijo de agentes vendedores (*seller1* y *seller2*) y que cada agente comprador ya los conoce (véase la variable miembro `AID[] sellerAgents` de la clase `BookBuyerAgent` en el código presentado en el apartado 3.4). En este capítulo describimos cómo prescindir de esta suposición y aprovechar el servicio de páginas amarillas que ofrece la plataforma JADE para que los agentes compradores descubran dinámicamente a los agentes vendedores disponibles en un momento dado.

6.1 El agente DF

Un servicio de «páginas amarillas» permite a los agentes publicar uno o más servicios que ofrecen, de modo que otros agentes puedan encontrarlos y aprovecharlos sucesivamente, tal y como se describe en la figura 4.

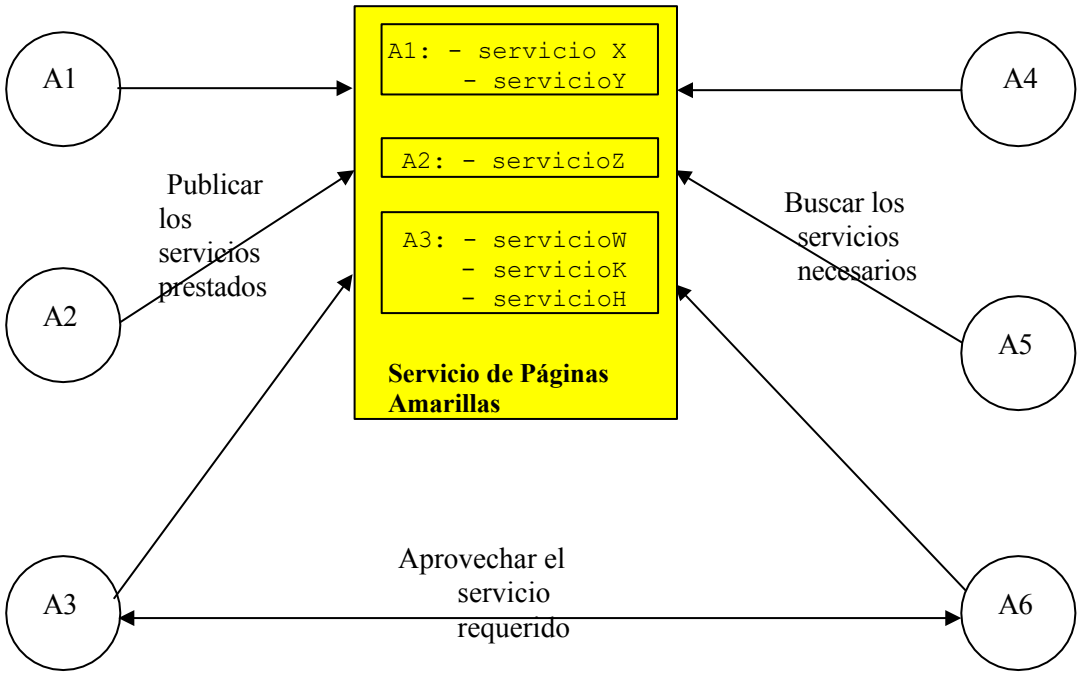


Figura 4. El servicio de Páginas Amarillas

El servicio de las Páginas Amarillas en JADE (según la especificación FIPA) lo proporciona un agente denominado DF (Directory Facilitator). Cada plataforma compatible con FIPA aloja un agente DF por defecto (cuyo nombre local es «df»). Se pueden activar otros agentes DF y se pueden federar varios agentes DF (incluido el por defecto) para proporcionar un único catálogo distribuido de las Páginas Amarillas.

6.2 Interacción con el DF

Dado que el DF es un agente, es posible interactuar con él como de costumbre mediante el intercambio de mensajes ACL utilizando un lenguaje de contenido adecuado (el lenguaje SL0) y una ontología adecuada (la ontología FIPA-agent-management), de acuerdo con la especificación FIPA. Sin embargo, para simplificar estas interacciones, JADE proporciona la clase `` `jade.domain.DFService` ``, mediante la cual es posible publicar y buscar servicios a través de llamadas a métodos.

6.2.1 Publicación de servicios

Un agente que desee publicar uno o más servicios debe proporcionar al DF una descripción que incluya su

AID, posiblemente la lista de lenguajes y ontologías que otros agentes necesitan conocer para interactuar con él, y la lista de servicios publicados. Para cada servicio publicado se proporciona una descripción que incluye el tipo de servicio, el nombre del servicio, los lenguajes y ontologías necesarios para utilizar dicho servicio y una serie de propiedades específicas del servicio. Las clases `DFAgentDescription`, `ServiceDescription` y `Property`, incluidas en el paquete `jade.domain.FIPAAgentManagement`, representan las tres abstracciones mencionadas.

Para publicar un servicio, un agente debe crear una descripción adecuada (como una instancia de la clase `DFAgentDescription`) y llamar al método estático `register()` de la clase `DFService`. Normalmente, aunque no necesariamente, el registro (publicación) del servicio se realiza en el método `setup()`, tal y como se muestra a continuación en el caso del agente «Book seller».

```
protected void setup() {
  ...
  // Registrar el servicio de venta de libros en las páginas amarillas
  DFAgentDescription dfd = new DFAgentDescription();
  dfd.setName(getAID());
  ServiceDescription sd = new ServiceDescription();
  sd.setType("book-selling");
  sd.setName(«JADE-book-trading»);
  dfd.addServices(sd);
  try {
    DFService.register(this, dfd);
  }
  catch (FIPAException fe) {
    fe.printStackTrace();
  }
  ...
}
```

Ten en cuenta que en este sencillo ejemplo no especificamos ninguna propiedad específica del lenguaje, la ontología o el servicio. Cuando un agente finaliza su ejecución, es recomendable dar de baja los servicios publicados.

```
protected void takeDown() {
  // Darse de baja de las páginas
  amarillas try {
    DFService.deregister(this);
  }
  catch (FIPAException fe) {
    fe.printStackTrace();
  }
  // Cerrar la interfaz
  gráfica de usuario
  myGui.dispose();
  // Mostrar un mensaje de cierre
  System.out.println(«Agente vendedor "+getAID().getName()+" finalizando.»);
}
```

6.2.2 Búsqueda de servicios

Un agente que desee buscar servicios debe proporcionar al DF una descripción de plantilla. El resultado de la búsqueda es la lista de todas las descripciones que coinciden con la plantilla proporcionada. Una descripción coincide con la plantilla si todos los campos especificados en la plantilla están presentes en la descripción con los mismos valores.

El método estático `search()` de la clase `DFService` se puede utilizar tal y como se muestra en el código empleado por el agente comprador de libros para encontrar dinámicamente todos los agentes que ofrecen un servicio de tipo «venta de libros».

```
public class BookBuyerAgent extends Agent {
  // El título del libro que se va a
  comprar private String targetBookTitle;
  // La lista de agentes vendedores
  conocidos private AID[] sellerAgents;
```

```
// Coloca aquí las inicializaciones del
agente protected void setup() {
  // Imprime un mensaje de bienvenida
  System.out.println("¡Hola! El agente comprador "+getAID().getName()+" está listo.");

  // Obtener el título del libro que se va a comprar como
  argumento de inicio Object[] args = getArguments();
  if (args != null && args.length > 0) {
    targetBookTitle = (String) args[0];
    System.out.println(«Intentando comprar « + targetBookTitle);

    // Añadir un TickerBehaviour que programe una solicitud a los agentes vendedores cada
    minuto addBehaviour(new TickerBehaviour(this, 60000) {
      protected void onTick() {
        // Actualizar la lista de agentes vendedores
        DFAgentDescription template = new DFAgentDescription();
        ServiceDescription sd = new ServiceDescription();
        sd.setType("book-selling");
        plantilla.addServices(sd);
        try {
          DFAgentDescription[] result = DFService.search(myAgent, template);
          sellerAgents = new AID[result.length];
          for (int i = 0; i < result.length; ++i) {
            sellerAgents[i] = result[i].getName();
          }
        }
        catch (FIPAException fe) {
          fe.printStackTrace();
        }
        // Realizar la solicitud
        myAgent.addBehaviour(new RequestPerformer());
      }
    } );
    ...
```

Ten en cuenta que la actualización de la lista de agentes vendedores conocidos se realiza antes de cada intento de compra del libro en cuestión, ya que los agentes vendedores pueden aparecer y desaparecer dinámicamente en el sistema. La clase `DFService` también permite suscribirse al DF para recibir una notificación tan pronto como un agente publique un servicio determinado (véanse los métodos `searchUntilFound()` y `createSubscriptionMessage()`), pero esto queda fuera del alcance de este tutorial.